

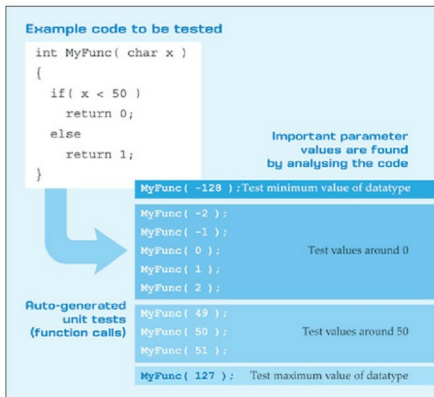


Fig. 1: A trivial C function can be tested by calling it many times with different input parameters. The selection of input parameter values is selected and generated automatically by the testing tool

is often exacerbated when delivery schedules become compressed with project progression. If source code and unit tests become out-of-sync, the tests will become less useful, especially when these are needed the most.

The unit test tools available to PC developers are less useful to embedded developers as these rarely manage compilation, downloading and execution of the test suites on embedded boards. Tools that create the unit tests automatically, build these into the embedded application and allow to run on the target via a convenient pathway, such as a joint test action group (JTAG) debugger, result in optimal usage of time and maximum productivity.

An effective approach is to use a fully-embedded test automation system that is integrated right into the C/C++ integrated development environment. Thus synchronisation is easy to maintain and ancillary tools such as JTAG debug probes can be

used to full effect. Such tools were not common previously, but new tools now meet these demanding criteria and bring powerful in-target test automation capabilities to software developers.

Unit test automatic generation.

The new breed of professional embedded test automation tools can analyse the source code and generate and

execute suitable test suites automatically on the target board. For example, these auto-generate a test suite and make many function calls with different combinations of input parameters, thus driving a large number of different execution paths.

Unit test automatic execution.

Once the test suite has been generated, it must be compiled, linked and executed on the target system. Many low-cost unit test tools exist that run unit tests only on a Windows PC. The inability of these tools to integrate with the embedded tool development environment results in additional inefficiency to execute the tests on the target board. However, the new breed of professional embedded test automation tools integrate into a professional embedded IDE, enabling easy synchronisation of test with code development. These also take advantage of integration with tools such as JTAG debug probe.

Measuring test quality

Once a code has been developed and tested, the focus shifts to quantitatively understanding what actually happened during testing. Code-coverage measurement, which is performed using dynamic execution flow analysis, is commonly used to study which parts of the code have been tested.

There are many different types of code-coverage analysis—from the very simple to very stringent types. Some types are:

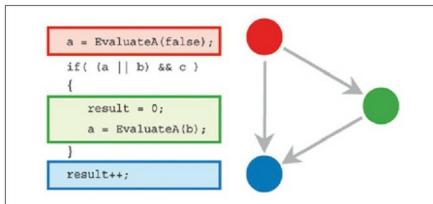


Fig. 2: The branch decision in the if-statement drives the selection of one out of the two possible execution paths—either directly from the red to the blue code block; or passing from red to blue via the green code block